

Algoritmi 2

*I seguenti appunti sono a cura di **Catalin Ceban** e si basano sul corso di "Algoritmi 2" erogato nell'A.A. 2026 dal professore Paul Joseph Wollan per il C.d.L Informatica (33503)*

25 settembre 2026

Indice

1	Introduzione e Teoria dei Grafi	2
1.1	Argomenti principali	2
1.2	Definizioni preliminari sui grafi	2
2	Introduzione ai grafi e passeggiate euleriane	4
2.1	Il problema dei ponti di Königsberg	4
3	Rappresentazione dei grafi	6
3.1	Rappresentazione fisica dei grafi	6
3.1.1	Matrici di adiacenza	6
3.1.2	Liste di adiacenza	7
4	Passeggiate e Cammini nei Grafi	8
5	Ricerca di cicli nei grafi	10
6	Ricerca di cicli nei grafi	12
7	Rilevamento di cicli nei grafi	14
8	Analisi delle versioni	16

Capitolo 1

Introduzione e Teoria dei Grafi

In questa sezione introduttiva delineiamo gli argomenti principali che saranno trattati nel corso, ponendo le basi per lo studio strutturato degli algoritmi avanzati.

1.1 Argomenti principali

Il corso si articolerà principalmente attorno ai seguenti nuclei tematici:

1. **Grafi e algoritmi sui grafi:** studio delle strutture dati fondamentali per la rappresentazione di relazioni e delle relative tecniche di visita ed elaborazione.
2. **Algoritmi greedy:** strategie di scelta ottima locale per la risoluzione di problemi globali (ad esempio, la ricerca di un albero di copertura di peso minimo connettendo vari nodi).
3. **Divide et impera:** paradigma algoritmico basato sull'approccio *top-down*, che consiste nel suddividere un problema in sottoproblemi più piccoli, risolverli ricorsivamente e combinarne le soluzioni.
4. **Programmazione dinamica:** approccio di tipo *bottom-up* per la risoluzione di problemi caratterizzati da sottoproblemi sovrapposti, memorizzando i risultati parziali per evitare ricalcoli superflui.

Osservazione 1.1.1: Albero

Ricordiamo che un **albero** è definito formalmente come un grafo connesso e aciclico.

1.2 Definizioni preliminari sui grafi

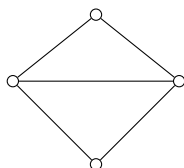
Un **grafo** è una coppia $G = (V, E)$, dove:

- V rappresenta l'insieme dei vertici o nodi.

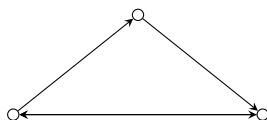
- E rappresenta l'insieme degli archi o lati.

Per comodità di notazione, nel corso del manuale denoteremo con $n = |V|$ il numero dei vertici e con $m = |E|$ il numero degli archi.

Ogni arco è una coppia di punti che indica i due vertici collegati. A titolo esemplificativo, consideriamo la seguente rappresentazione grafica:



Riguardo agli archi, potremmo avere anche un orientamento: se ogni arco possiede una direzione definita, diciamo che il grafo è **diretto**.



È importante notare che possiamo avere anche due archi sugli stessi nodi ma con orientamento diverso (come mostrato nel collegamento bidirezionale o nei doppi archi orientati tra i medesimi vertici).

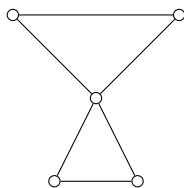
Sia f un arco che collega due vertici x e y , formalmente rappresentabile come $x \leftarrow f \rightarrow y$. Diciamo che:

- L'arco f è **incidente** ai vertici x e y .
- Il vertice x è **adiacente** al vertice y .

Definizione 1.2.1: Grado di un vertice

Il **grado** di un vertice x , denotato con $\deg(x)$, è il numero di archi incidenti in x .

Ad esempio, considerando un nodo centrale collegato a quattro vertici periferici:



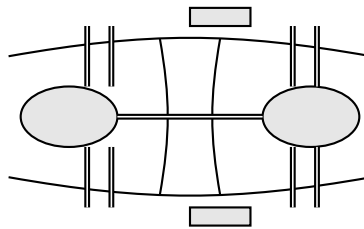
Per il nodo centrale x contrassegnato, avremo chiaramente $\deg(x) = 4$, poiché vi incidono esattamente quattro archi distinti.

Capitolo 2

Introduzione ai grafi e passeggiate euleriane

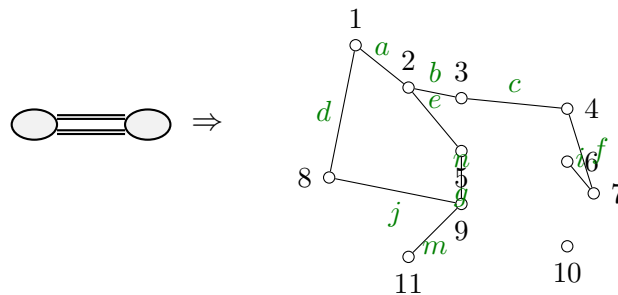
2.1 Il problema dei ponti di Königsberg

Il problema classico da cui nasce la teoria dei grafi è il famoso problema dei ponti di Königsberg:



Ci si chiede: *è possibile attraversare tutti i ponti una sola volta e arrivare al punto di partenza?*

Traduciamo il problema in un problema di grafi:



Definizione 2.1.1: Passeggiata in un grafo

Una **passeggiata** in un grafo è una sequenza $v_0, e_1, v_1, e_2, v_2, \dots, v_{k-1}, e_k, v_k$ che alterna vertici e archi.

Definizione 2.1.2: Passeggiata chiusa

Una passeggiata si dice **chiusa** se $v_0 = v_k$, cioè il punto di inizio coincide con il punto di arrivo.

Qualsiasi grafo con almeno un nodo x con $\deg(x)$ dispari **non** ammette una passeggiata chiusa che utilizza tutti gli archi una sola volta. In questo caso, dunque: **no**, non è risolvibile il problema.

E se fossero tutti pari? **Non è detto**.

$\diagup \diagdown$
—

—
 $\diagdown \diagup$

Tutti $\deg = 2$, ma non essendo connesso non è possibile.

Definizione 2.1.3: Grafo connesso

Un grafo si dice **connesso** se per ogni coppia di vertici esiste una passeggiata da x a y .

Capitolo 3

Rappresentazione dei grafi

Teorema 3.0.1: Caratterizzazione di Eulero

Esiste una passeggiata chiusa che contiene ogni arco una sola volta se e solo se:

1. Ogni vertice ha grado pari.
2. Il grafo è connesso.

3.1 Rappresentazione fisica dei grafi

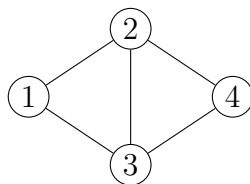
La rappresentazione informatica dei grafi in memoria è fondamentale per determinare l'efficienza degli algoritmi che operano su di essi. Esistono principalmente due modalità standard per rappresentare un grafo: le matrici di adiacenza e le liste di adiacenza.

3.1.1 Matrici di adiacenza

Si stabilisce una matrice $n \times n$ dove $n = \#V$, e dove:

$$m_{i,j} = \begin{cases} 1 & \text{se } i \text{ è adiacente a } j \\ 0 & \text{altrimenti} \end{cases}$$

Esempio:



$$\begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

Se il grafo non è diretto, la matrice sarà simmetrica rispetto alla diagonale.

Costo computazionale:

- Visita del grafo e memorizzazione: $\Theta(n^2)$.
- Per verificare se i è adiacente a j : $\Theta(1)$.
- Verificare $\deg(i)$: $\Theta(n)$ (occorre scorrere la riga i -esima e contare quanti 1 ci sono).

3.1.2 Liste di adiacenza

Una lista di adiacenza è composta da un vettore di teste di liste concatenate. Ciascuna lista contiene i nodi adiacenti al rispettivo vertice.

$$\begin{array}{l} \left[\begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \end{array} \right] \rightarrow \begin{array}{l} [2, 3] \\ [1, 3, 4] \\ [1, 2, 4] \\ [2, 3] \end{array} \end{array}$$

Il motivo per cui lo spazio complessivo dipende dal numero di archi è che ogni arco viene contato due volte, una per ogni nodo coinvolto nel grafo non orientato.

Costo computazionale (C.C.):

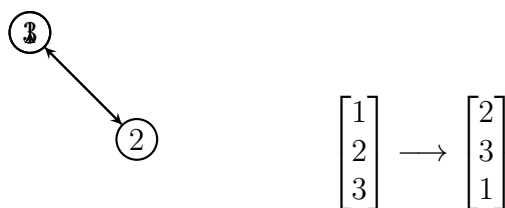
- Per verificare se i è adiacente a j : $\mathcal{O}(\deg(i))$.
- Per visitare e memorizzare: $\mathcal{O}(n + \sum_v \deg(v)) = \mathcal{O}(n + 2m) = \mathcal{O}(n + m)$, dove $n = \#V$ e $m = \#E$.

L'occupazione di memoria $\mathcal{O}(n)$ è dovuta chiaramente al fatto che deve esserci un array con un elemento per ogni nodo. Ogni nodo poi dovrà avere un puntatore ad una lista concatenata che rappresenta i suoi adiacenti.

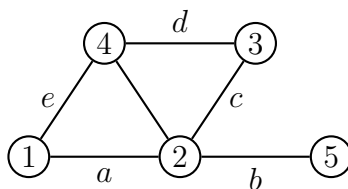
Capitolo 4

Passeggiate e Cammini nei Grafi

Ci poniamo ora la seguente domanda: *come si estendono questi concetti nel caso dei grafi diretti?*



C'è un altro modo per memorizzare questi concetti. Consideriamo un grafo non orientato con i vertici 1, 2, 3, 4, 5 collegati da vari archi etichettati.



Una passeggiata all'interno del grafo potrebbe essere la seguente sequenza:

$$1 \xrightarrow{a} 2 \xrightarrow{c} 3 \xrightarrow{d} 4 \xrightarrow{e} 2 \xrightarrow{b} 5$$

Definizione 4.0.1: Cammino

Un **cammino** in un grafo è una passeggiata che non ripete i vertici.

La passeggiata vista prima **non** è un cammino da 1 a 5, ma un possibile cammino tra 1 e 5 potrebbe essere:

$$1 \xrightarrow{a} 2 \xrightarrow{b} 5$$

Proposizione 4.0.1: Esistenza di cammini e passeggiate

Esiste una passeggiata da x a y se e solo se esiste un cammino da x a y .

Dimostrazione 4.0.1

Chiaramente, se esiste un cammino allora sicuramente esiste anche una passeggiata, dato che ogni cammino è un particolare tipo di passeggiata.

Viceversa, supponiamo di avere una passeggiata $W = (v_0, v_1, \dots, v_K)$. Abbiamo due possibilità:

1. Se tutti i vertici sono distinti, allora la passeggiata è già un cammino.
2. Se invece abbiamo almeno un indice ripetuto, cioè esistono i, j tali che $v_i = v_j$ con $i < j$.

Questo significa che all'interno della passeggiata è presente un sottociclo:

$$W_1 = (v_i, v_{i+1}, \dots, v_{j-1}, v_j)$$

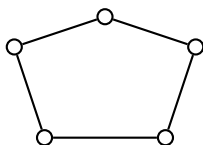
ma sapendo che $v_i = v_j$, la struttura si chiude. Possiamo eliminare W_1 dalla passeggiata iniziale mantenendo però il nodo in cui parte la diramazione (in questo caso v_i). La passeggiata iniziale può essere dunque riscritta come:

$$W = (v_1, \dots, v_i, \underbrace{v_{i+1}, \dots, v_{j-1}, v_j}_{W_1}, v_{j+1}, \dots, v_K)$$

Eliminando W_1 otteniamo un cammino più corto, e reiterando il procedimento rimuoviamo tutti i cicli ottenendo infine un cammino semplice.

Definizione 4.0.2: Ciclo

Un **ciclo** è un grafo connesso in cui ogni vertice ha grado 2.



Capitolo 5

Ricerca di cicli nei grafi

Infine la passeggiata diventa:

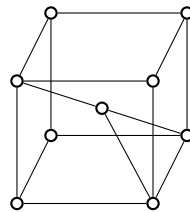
$$W = (v_1, v_2, \dots, v_i, v_{j+1}, \dots, v_k)$$

che, non contenendo nodi ripetuti, è un cammino.

Definizione 5.0.1: Problema del ciclo

Dato un grafo G dove per ogni vertice x si ha $\deg(x) \geq 2$, trovare un ciclo nel grafo.

Prociamo prima graficamente:



L'idea iniziale consiste nel:

- Prendere un punto di partenza.
- Prendere un arco e andare al nodo adiacente.
- Memorizzare i nodi visitati finora.
- Controllare se il prossimo nodo è stato già visitato.

Presentiamo di seguito l'algoritmo per la ricerca del ciclo all'interno del grafo G .

Spiegazione algoritmo 5.0.1: Algoritmo di ricerca del ciclo

Il procedimento riceve in input un grafo G in cui ogni vertice ha grado almeno 2. L'obiettivo è individuare un ciclo sfruttando una lista di supporto per tracciare i nodi visitati.

```
1      Ciclo(G):  
2      x = un vertice qualsiasi  
3      creo una LISTA PUNTATA vuota (L)  
4      append(x)  
5      teniamo traccia di current(x) e next = x.  
        neighbour[0]  
6      while next not in L:  
7      L.append(next)  
8      if next.neighb[0] == current:  
9      u = next.neighb[1]  
10     else:  
11     u = next.neighb[0]  
12     current = next  
13     next = u  
14     while L[0] != next:  
15     L.remove(0)  
16     return L
```

Note sull'implementazione e complessità:

- La ricerca del nodo all'interno della lista L ('while next not in L') richiede normalmente $O(n)$, ma tale operazione può essere gestita facilmente definendo un vettore di supporto 'vis = [0] * n' che si aggiorna ponendo 'vis[next] = 1' ad ogni iterazione, riducendo il costo di controllo a $O(1)$.
- Se invece si decidesse di utilizzare una matrice di adiacenza, il controllo costerebbe comunque $O(n)$, portando il costo computazionale complessivo a $O(n^2)$.

Capitolo 6

Ricerca di cicli nei grafi

L'idea di base dell'algoritmo risolutivo per la ricerca di un ciclo all'interno di un grafo (supportata dall'ipotesi forte che ogni vertice abbia grado $\deg(x) \geq 2$) si articola nei seguenti punti:

1. Si sceglie un nodo di partenza x e lo si memorizza in una struttura dati che permetta di tenere traccia dei nodi visitati.
2. Si prende un qualsiasi nodo adiacente, controllando categoricamente che non sia già stato visitato (per la prima iterazione è impossibile, ma in generale...).

Siamo certi che esista un ciclo proprio grazie all'ipotesi forte di $\deg(x) \geq 2$ per ogni x . Proviamo a formalizzare il tutto sotto forma di codice.

```
1  def trovaCiclo(G):
2  L = lista vuota
3  x = vertice qualsiasi
4  L.append(x)
5  Current = x
6  next = x.neighbr[0]
7
8  while (next not in L):
9  L.append(next)
10 if next.neighbr[0] == current:
11 u = next.neighbr[1]
12 else:
13 u = next.neighbr[0]
14 current = next
15 next = u
16
17 while L[0] != next:
18 L.remove(0)
19
20 return L
```

Spiegazione algoritmo 6.0.1: Analisi e spiegazione dell'algoritmo

Il codice presentato implementa la ricerca di un ciclo all'interno del grafo G . Analizziamo dettagliatamente le singole componenti e le istruzioni evidenziate:

- **Inizializzazione:** Viene creata una lista vuota L per tracciare il percorso e viene scelto un vertice iniziale x aggiunto alla lista. Si impostano i puntatori iniziali `current` e `next`. Nota importante: l'operazione di controllo di presenza all'interno di una lista non ordinata (`next not in L`) richiede tempo $O(n)$, rendendo questa versione iniziale inefficiente.
- **Esplorazione (Righe 7-14):** Finché il nodo successivo `next` non è già presente in L^1 , il nodo viene aggiunto alla lista². Si controlla quale dei vicini corrisponde al nodo corrente per evitare di tornare indietro sui propri passi: se il primo vicino è il nodo corrente, si sceglie il secondo, altrimenti si prende tranquillamente il primo dei vicini.
- **Aggiornamento dei puntatori (Riga 12-13):** Vengono preparati i puntatori per la successiva iterazione³ aggiornando `current` a `next` e `next` a u . Questo ciclo viene eseguito al più n volte, portando a una complessità temporale complessiva di $T(n) = O(n^2)$.
- **Pulizia della struttura (Righe 16-17):** Una volta individuato il ciclo (quando `next` è già presente in L , il che significa che siamo in qualche modo tornati sui nostri passi), si rimuovono dalla lista L tutti gli elementi iniziali che non fanno parte del ciclo, finché il primo elemento di L non coincide con il nodo `next`.

Capitolo 7

Rilevamento di cicli nei grafi

Quando raggiungiamo un ciclo, la variabile `next` contiene l'identificativo del vertice su cui il ciclo si è chiuso. Tuttavia, la lista L contiene tutto il cammino percorso fino a quel momento, quindi dobbiamo eliminare tutto ciò che non fa parte del ciclo.

Controlliamo se $L[0] \neq \text{next}$: sapendo che `next` è l'ultimo nodo del ciclo, significa che esso è anche il primo nodo del ciclo stesso. Dunque, il primo valore della lista dovrà essere proprio `next`, in quanto è il punto da cui tutto parte.

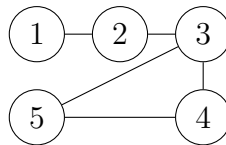


Figura 7.1: Esempio grafico di un grafo con un ciclo.

Dopo il primo ciclo, abbiamo $L = [1, 2, 3, 4, 5]$, `current` = 5 e `next` = 3. Inizia il secondo ciclo di pulizia della lista:

- $L[0] \neq 3$? Sì, rimuoviamo 1 e otteniamo $L = [2, 3, 4, 5]$.
- $L[0] \neq 3$? Sì, rimuoviamo 2 e otteniamo $L = [3, 4, 5]$.
- $L[0] \neq 3$? No, il primo elemento è 3: ciclo trovato, fine.

Tuttavia, come già detto, il contro di questa versione è che il controllo di presenza di un nodo nella lista può essere dispendioso. Possiamo infatti mitigare quel costo con l'uso di una lista `vis` che mappa i nodi visitati.

Spiegazione algoritmo 7.0.1: Algoritmo di ricerca del ciclo con vettore dei visitati

Viene presentata di seguito l'implementazione ottimizzata per la ricerca di un ciclo all'interno di un grafo G , utilizzando un vettore di supporto per tracciare i nodi visitati in tempo costante $\mathcal{O}(1)$.

```

1      def trovaCiclo(G):
2          n = len(G.V)
3          L = lista()
4          vis = [0] * n
5
6          x = vertice_qualsiasi()
7          L.append(x) # Aggiungiamo alla cronologia
8          vis[x] = 1 # Segniamo come visitato
9
10         current = x
11         next = x.neighbr[0]
12
13         while vis[next] == 0: # Controllo in O(1)
14             L.append(next)
15             vis[next] = 1
16
17         if next.neighbr[0] == current:
18             u = next.neighbr[1]
19         else:
20             u = next.neighbr[0]
21
22         current = next
23         next = u
24
25         # Rimozione dei nodi esterni al ciclo (stessa
26             logica vista in precedenza)
27         while L[0] != next:
28             L.pop(0)
29
30         return L

```

Capitolo 8

Analisi delle versioni

Analizzando le due versioni, vediamo come:

- **Versione 1 (V1):**
 - Costo temporale: $O(n^2)$
 - Costo spaziale: $O(n)$, ottenuto mediante l'utilizzo di una lista L per tenere traccia della cronologia.
- **Versione 2 (V2):**
 - Costo temporale: $O(n)$
 - Costo spaziale: $O(n) + O(n) = O(2n) = O(n)$, dove i contributi spaziali sono dati rispettivamente dalla struttura L e dall'insieme di supporto per le visite Vis . Poiché il risultato si attesta sullo stesso ordine di grandezza, la complessità spaziale risulta essere ottima.